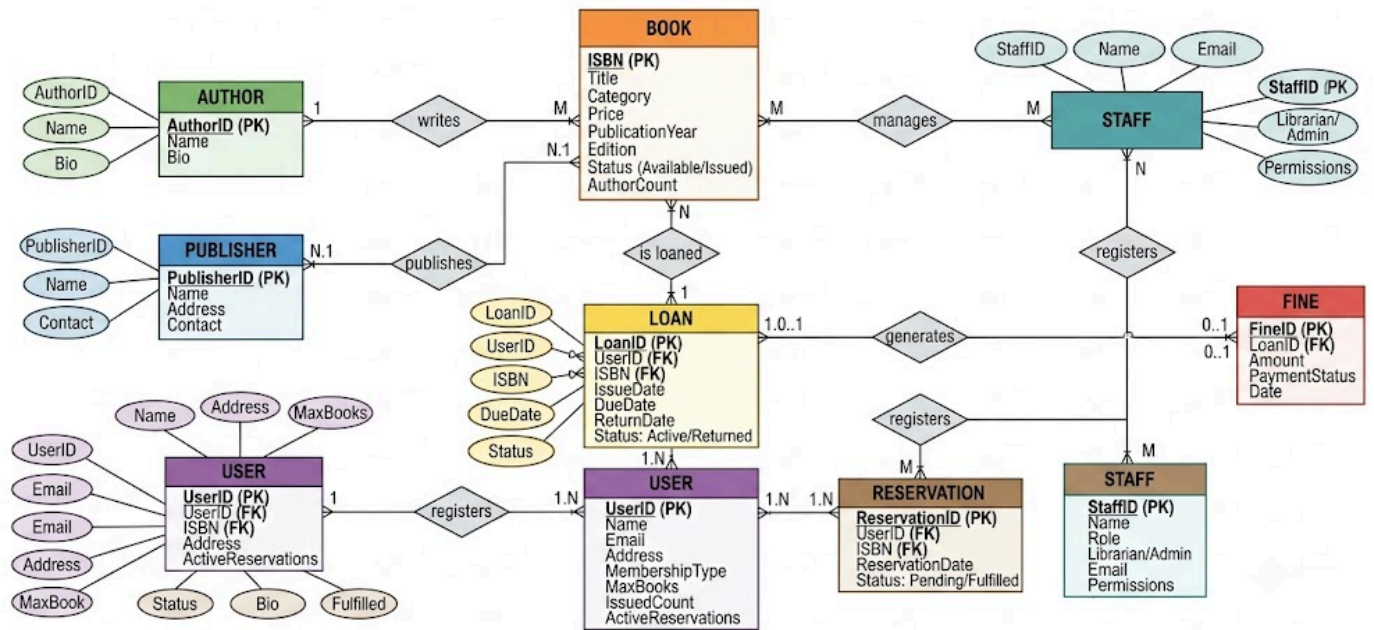


Topics to cover

1. Query writing or relational algebra
2. ER diagrams
 - a. Draw ER diagram
 - b. ER diagram to relation mapping
3. ACID property
4. RAID
 - a. Advantages
 - b. What happens when a disk fail
5. Indexing
 - a. Primary indexing
 - b. Secondary indexing
 - c. Sparse index
 - d. Dense index
 - e. Hash indexing
6. B+ tree indexing formation
 - a. Why its better than others
7. Trigger
8. Different types of keys
 - a. Foreign key
 - b. Primary key
 - c. Super key
 - d. Candidate key
9. Normalization
 - a. 1NF
 - b. 2NF
 - c. 3NF
 - d. BCNF
10. Concurrency control
 - a. Lock based control
 - b. Timestamp based control

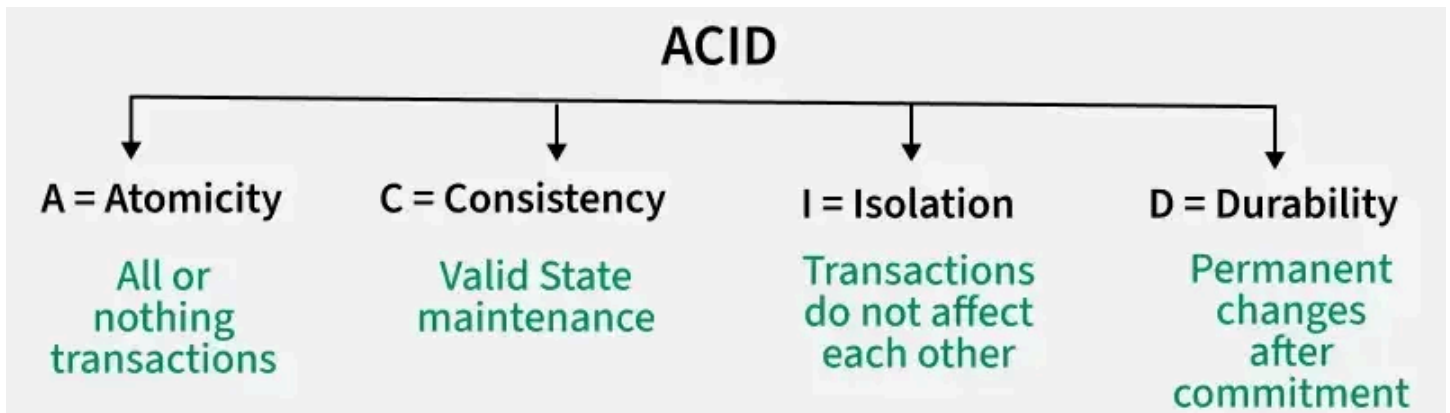
ER diagrams

LIBRARY MANAGEMENT SYSTEM - DETAILED ENTITY-RELATIONSHIP (ER) DIAGRAM



Legend: Entity (Green box), Weak Entity (Blue box), Attribute (Yellow oval), Key Attribute (Orange oval), Multivalued Attribute (Purple oval), Derived Attribute (Light Blue oval), Relationship (Diamond), Crow's Foot Notation (Line with crow's foot symbol)

ACID property



Example - Two customers are trying to withdraw money from the same joint bank account at the exact same millisecond using two different ATM booths. The account balance is 5,000 BDT. Customer A wants to withdraw 4,000 BDT and Customer B wants to withdraw 3,000 BDT. If both transactions are processed simultaneously without control, what problem will occur? How can you solve this using DBMS properties?

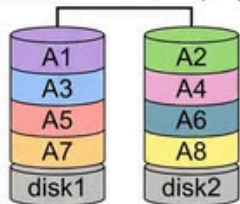
RAID

Why is RAID important for Databases?

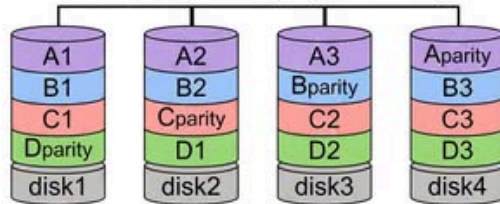
1. **Fault Tolerance:** Databases store - Transaction logs, Master data, Indexes, Constraints. If a single disk fails and there is no redundancy, then it can occur database corruption or complete downtime. RAID protects against disk failure by maintaining redundant copies or parity data.
2. **High Availability** - Keeps the database operational even if one disk crashes, minimizing downtime.
3. **Improved I/O Performance** - Uses striping to distribute read or write operations across multiple disks, increasing throughput.
4. **Protection of Transaction Logs** - Critical log files (WAL/redo logs) remain safe using mirroring (e.g., RAID 1 or RAID 10).
5. **Support for ACID Durability** - Ensures committed transactions are not lost due to hardware failure.

RAID Overview

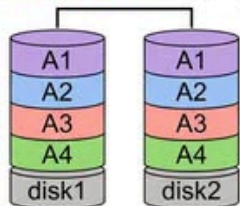
RAID Level 0: Striping



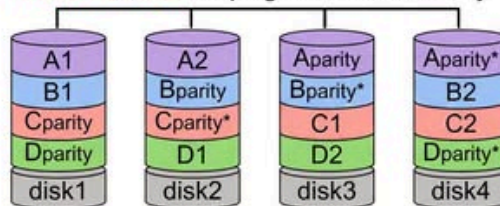
RAID Level 5: Striping with Parity



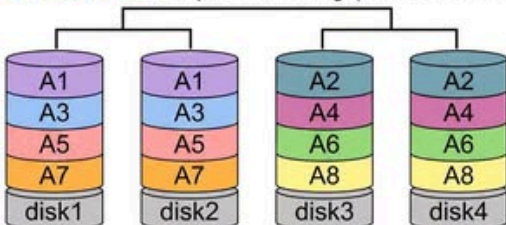
RAID Level 1: Mirroring



RAID Level 6: Striping with Double Parity



RAID Level 10: Striped Mirroring (aka RAID 1+0)



Sun
Networking
Academy

RAID level	Min disks	Available storage capacity (%)	Write penalty	Protection
1	2	50	2	Mirror
1+0	4	50	2	Mirror
3	3	$[(n-1)/n]*100$	4	Parity (Supports single disk failure)
5	3	$[(n-1)/n]*100$	4	Parity (Supports single disk failure)
6	4	$[(n-2)/n]*100$	6	Parity (Supports two disk failures)

Normalization

1NF

Students

FirstName	LastName	Knowledge
Thomas	Mueller	Java, C++, PHP
Ursula	Meier	PHP, Java
Igor	Mueller	C++, Java

Startsituation

Result after Normalisation



Students

FirstName	LastName	Knowledge
Thomas	Mueller	C++
Thomas	Mueller	PHP
Thomas	Mueller	Java
Ursula	Meier	Java
Ursula	Meier	PHP
Igor	Mueller	Java
Igor	Mueller	C++

2NF -

- Must be in 1NF: The table must first satisfy all the requirements of First Normal Form (1NF).
- No Partial Dependency: No non-key attribute should depend on only a part of a composite candidate key.

Students

IDSt	LastName	IDProf	Prof	Grade
1	Mueller	3	Schmid	5
2	Meier	2	Borner	4
3	Tobler	1	Bernasconi	6

Startsituation



Result after normalisation

Students

ID	LastName
1	Mueller
2	Meier
3	Tobler

Professors

IDProf	Professor
1	Bernasconi
2	Borner
3	Schmid

Grades

IDSt	IDProf	Grade
1	3	5
2	2	4
3	1	6

3NF -

1. It is in Second Normal Form (2NF): This means the table has no partial dependencies (i.e., no non-prime attribute is dependent on a part of a candidate key).
2. There is no transitive dependency for non-prime attributes: In simpler terms, no non-key attribute should depend on another non-key attribute. Instead, all non-key attributes should depend directly on the primary key.

Vendor

ID	Name	Account_No	Bank_Code_No	Bank
----	------	------------	--------------	------

Startsituation



Result after normalisation

Vendor

ID	Name	Account_No	Bank_Code_No
----	------	------------	--------------

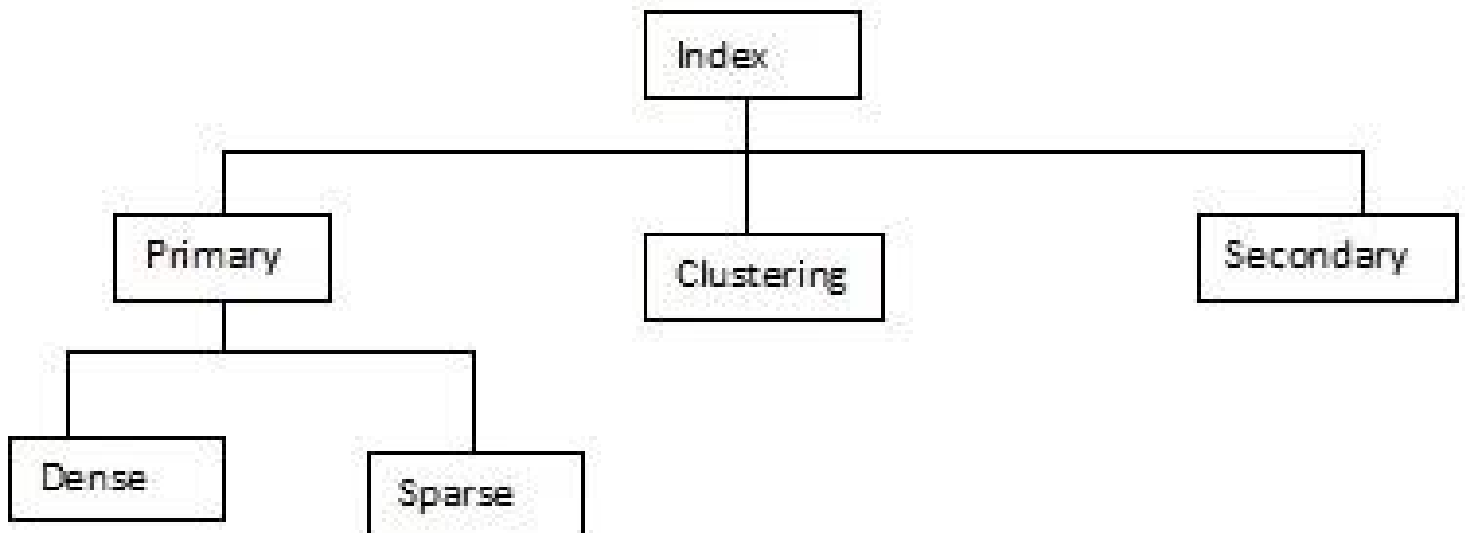
Bank

Bank_Code_No	Bank
--------------	------

3.5 or BCNF -

- Rule 1: The table should be in the 3rd Normal Form.
- Rule 2: X should be a super-key for every functional dependency (FD) $X \rightarrow Y$ in a given relation.

Indexing



B+ tree indexing formation

A B+ Tree is a balanced tree data structure widely used in database systems and file systems for efficient indexing and searching.

- **Balanced Tree** - All leaf nodes are at the same level. Thus the path length from the root to any leaf is equal.
- **Data Stored in Leaf Nodes** - Actual records or record pointers are stored only in the leaf nodes. Internal (non-leaf) nodes contain only keys and pointers.
- **Sorted Keys** - Keys are maintained in sorted order.
- **Leaf Nodes Are Linked** - Leaf nodes are connected using pointers.
- **Dynamic Growth and Shrink** - Nodes split when they become full. Nodes merge or redistribute keys when they become too empty. Tree remains balanced automatically.

Example - নিচের ডাটার জন্য B+ tree টি অংকন করে দেখান, যেখানে, $n = 4$:
1, 4, 7, 10, 17, 21, 31, 25, 19, 20, 28, 42

Trigger

Imagine you want to automatically log a record whenever a new order is placed.

```
CREATE TRIGGER AfterOrderInsert
AFTER
INSERT ON Orders
FOR EACH ROW
BEGIN
    INSERT INTO Order_Logs (OrderID, LogDate, Action, totalPrice)
    VALUES (NEW.OrderID, NOW(), 'New Order Placed', OLD.order_quantity * OLD.order_price);
END;
```

Concurrency control

